

AIに爆速で正確な図を書かせる

MDD(Markdown with Diagrams)を使ってAIにいい感じの図を作らせる技術

図の生成はすごく便利

認知負荷を下げるために、テキストだけでは伝わりにくい情報を図にすると便利。

設計書にアーキテクチャ図を入れたい



エンジニア

提案書にフロー図で流れを見せたい



PM

仕様書に画面遷移図を付けたい



デザイナー

しかし、AIで図の生成は課題が多い

AIが思ったとおりに図を作ってくれない



悩み1

AIが1つの図を作るだけで数分かかる



悩み2

AIが作った図はバージョン管理が難しい



悩み3

そもそも

Q AIに作図させるのはなぜこんなにも難しいのか？

A AIが情報を扱う上でグラフィカルに関わる情報は全てノイズ。
pptxやdrawioなどの図をしまうデータというのは
AIにとっては不要な情報の塊でしかない。

Q ではどうすればAIは図を上手に扱えるようになるか？

A AIに図を書かせようという発想がそもそもの間違い！
図を書くのはAIの仕事ではない！ AIはテキストデータ
だけを扱うべき。
図が欲しいならDSLを図に変換するツールをAIに作らせた
方が効率がいい！

ということで作りました。

 [ppdx999](#) / [mdd](#)

Markdown with Diagrams — テキストから図を生成する軽量プリプロセッサ

 Rust  MIT

<https://github.com/ppdx999/mdd>

MDD とは？

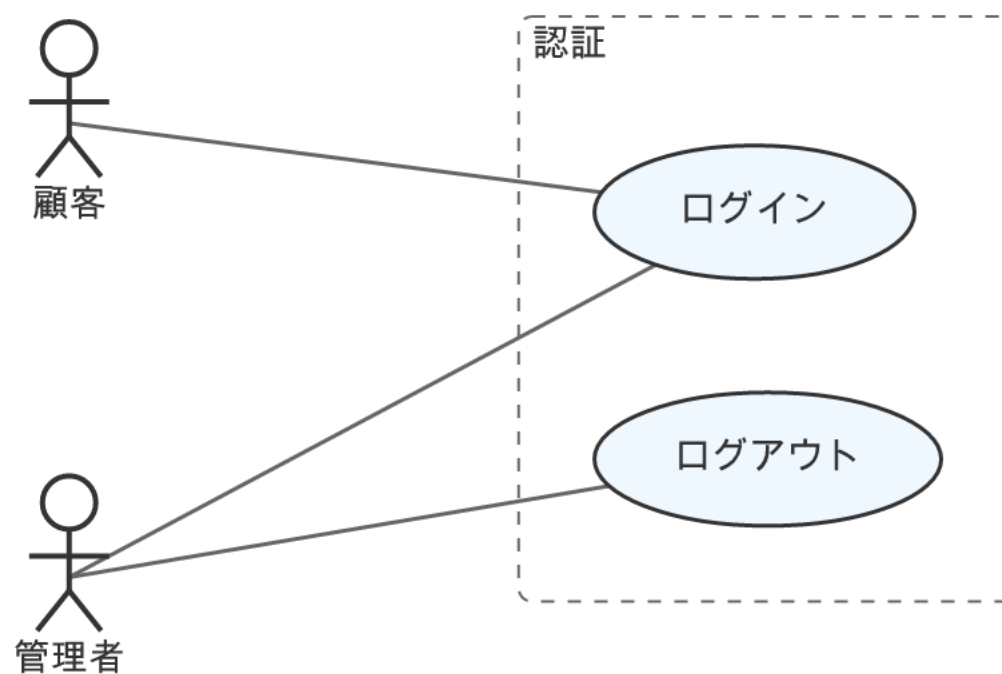
テキストからいい感じのSVGを生成するプログラム

例えば、以下のようなテキストをMDDに与えると...

```
1 actor 顧客
2 actor 管理者
3
4 package "認証" {
5   usecase ログイン
6   usecase ログアウト
7 }
8
9 顧客 -> ログイン
10 管理者 -> ログイン
11 管理者 -> ログアウト
```

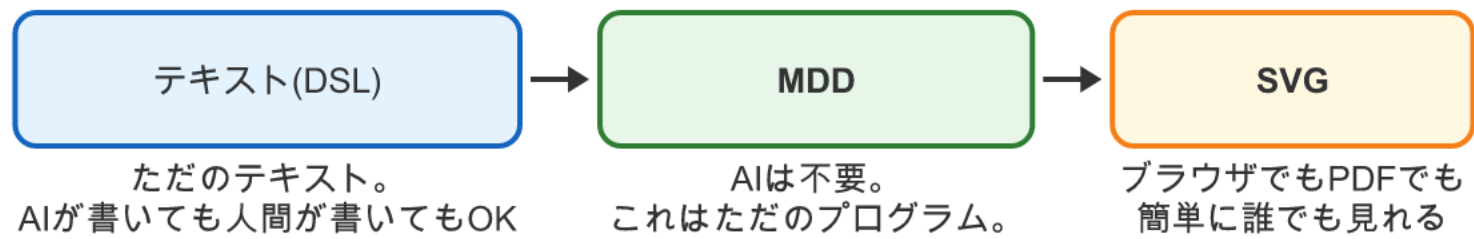


これをmddが変換すると...



いい感じのユースケース図(SVG)が出力される!

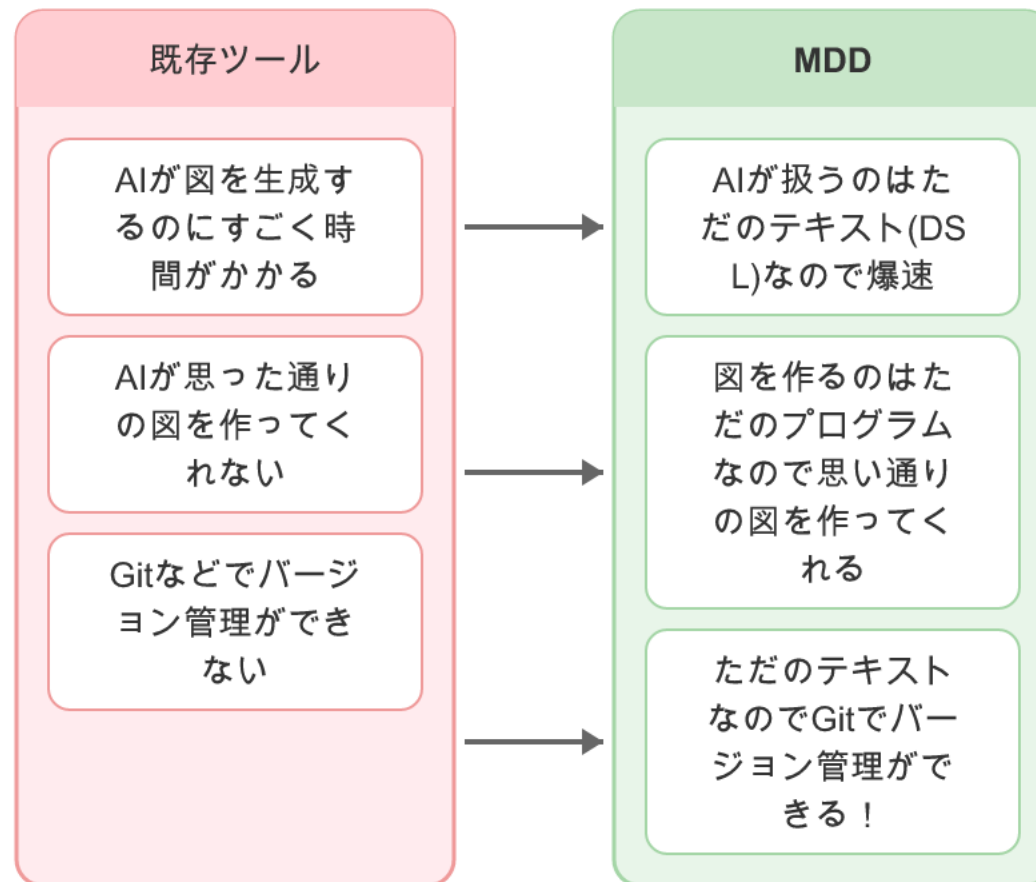
MDD の仕組み



余談

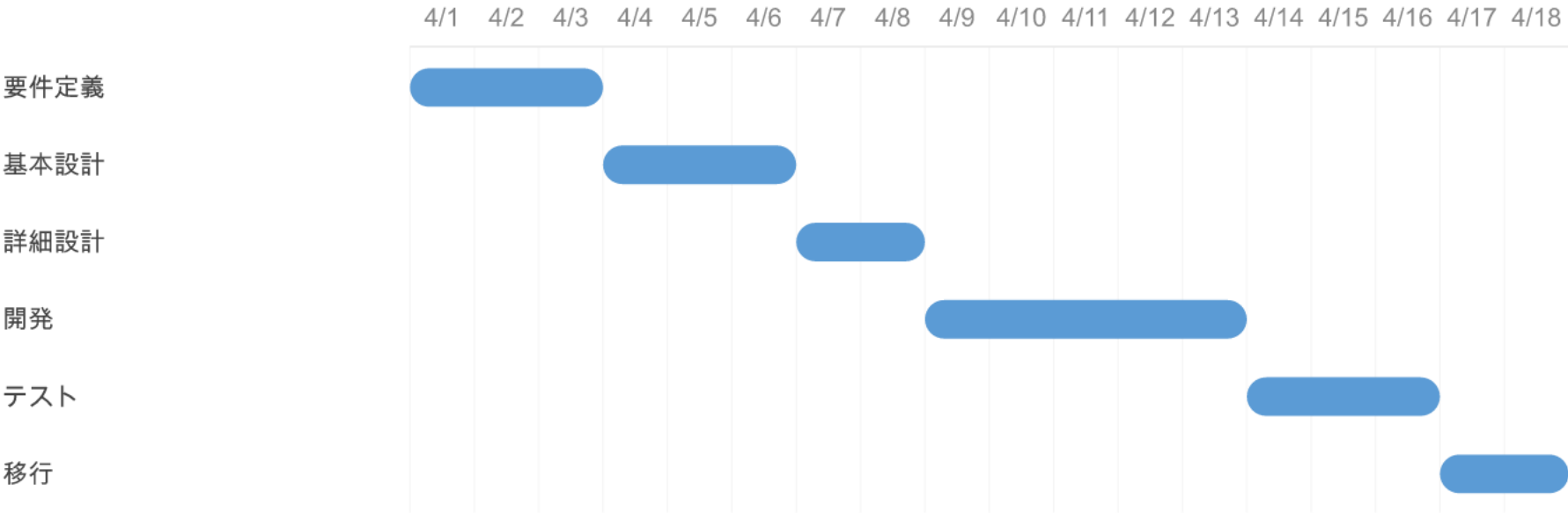
このテキストから図を作る技術は1981年に出させた論文の技術が元になっています。詳しく知りたい人はsugiyama法で検索！

この仕組みがなぜ良いのか？

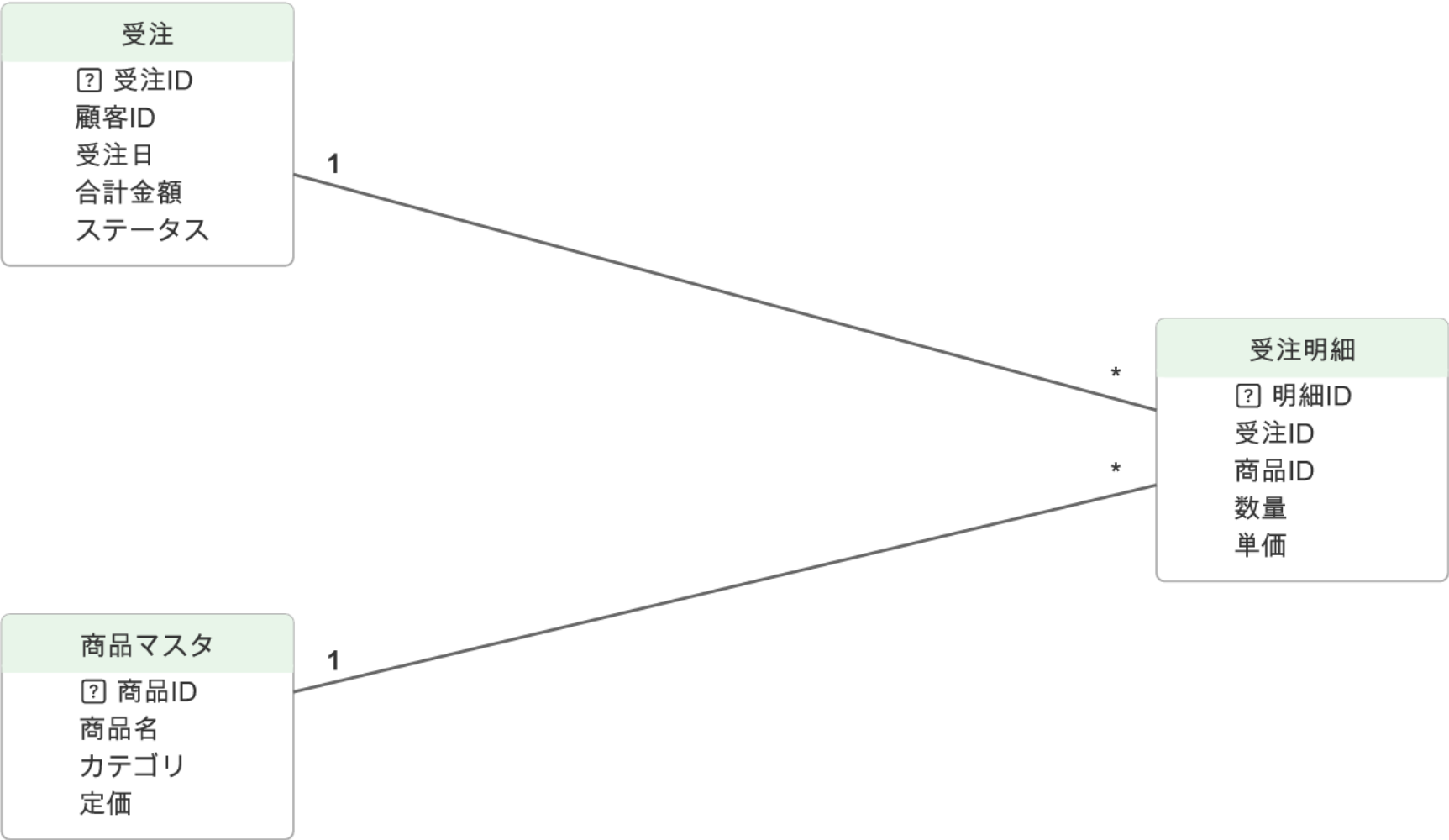


どんな図が作れる？ — プロジェクト計画

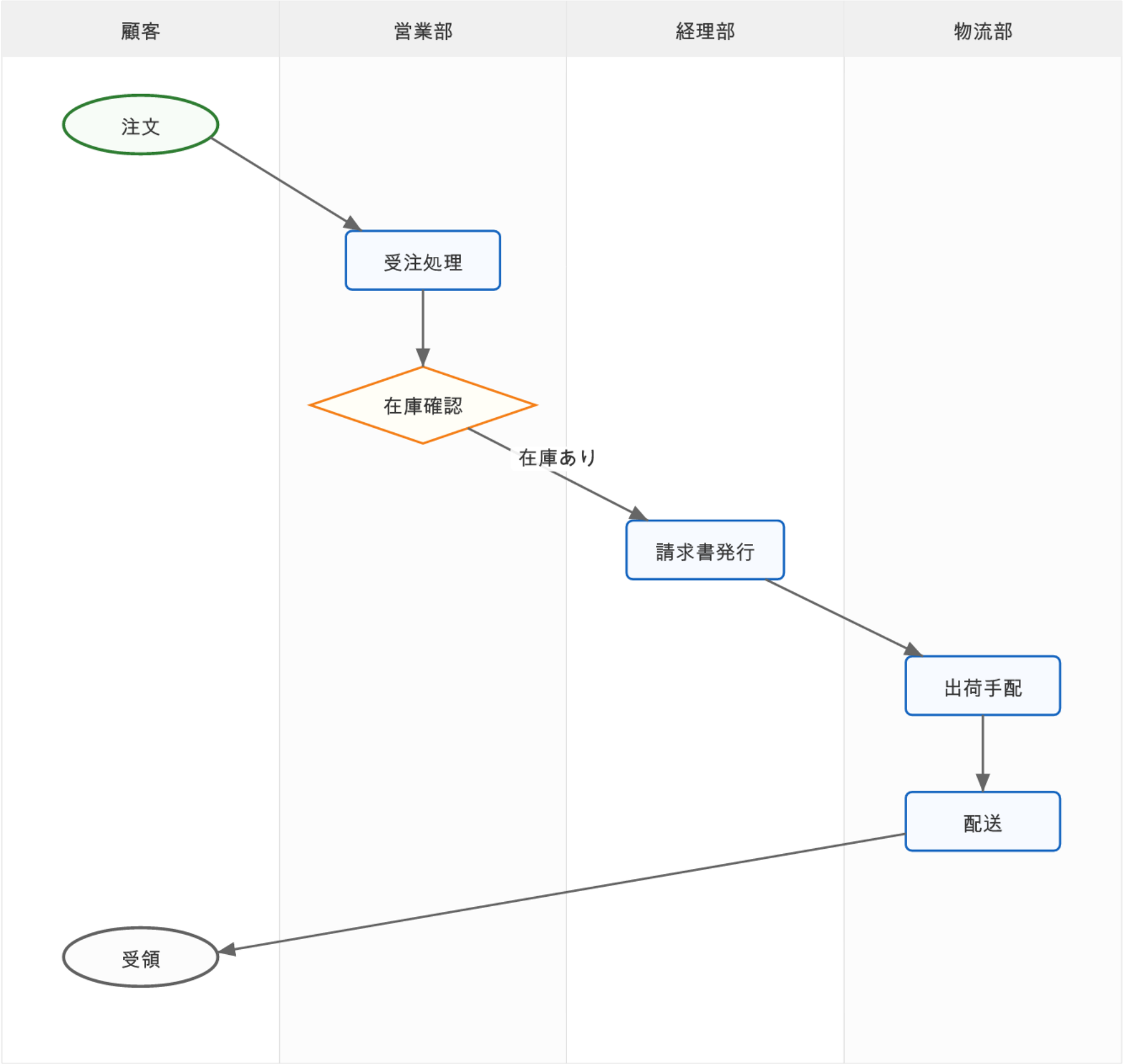
1	unit day
2	
3	要件定義 : 2025-04-01, 3d
4	基本設計 : after 要件定義, 3d
5	詳細設計 : after 基本設計, 2d
6	開発 : after 詳細設計, 5d
7	テスト : after 開発, 3d
8	移行 : after テスト, 2d



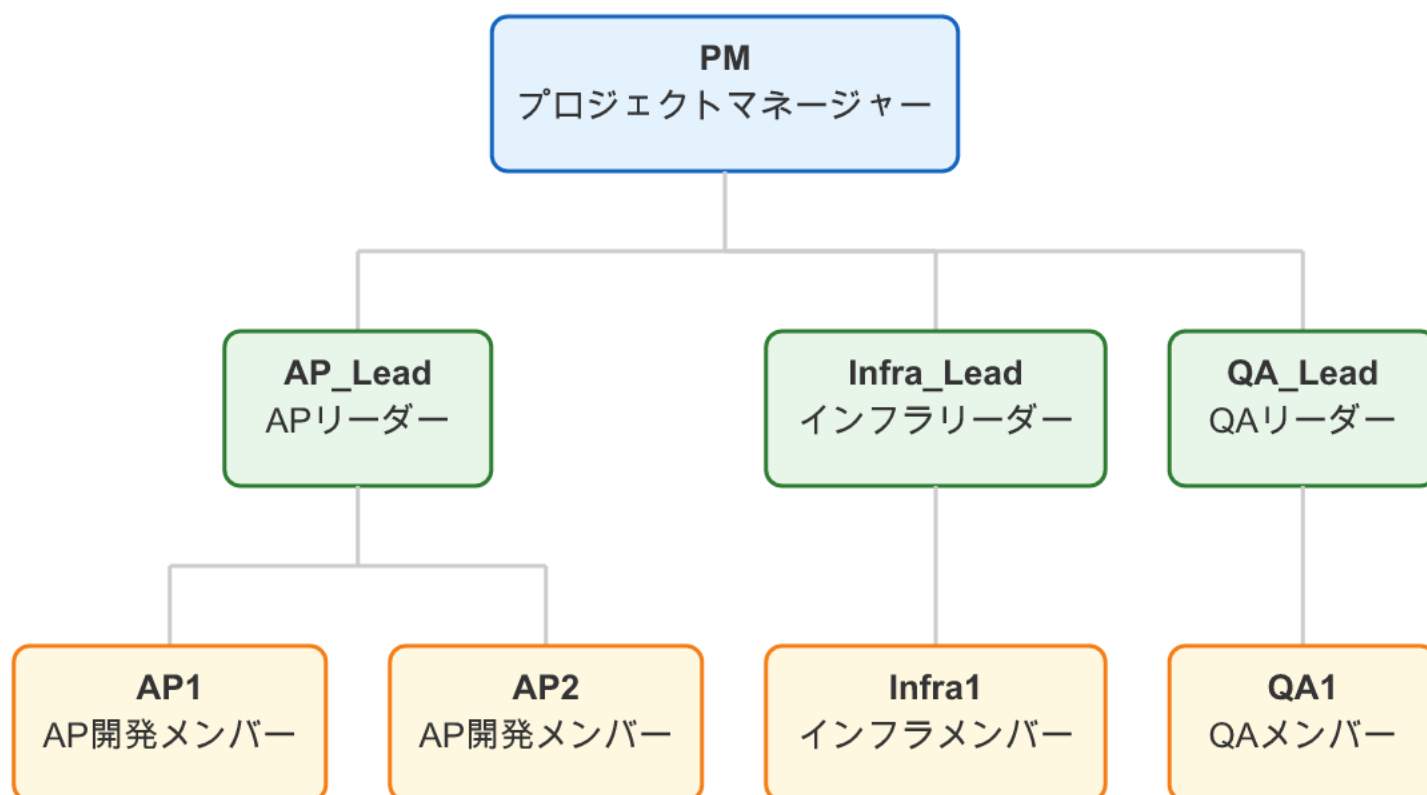
```
1  table 受注 {
2    * 受注ID
3    顧客ID
4    受注日
5    合計金額
6    ステータス
7  }
8
9  table 受注明細 {
10   * 明細ID
11   受注ID
12   商品ID
13   数量
14   単価
15 }
16
17 table 商品マスタ {
18   * 商品ID
19   商品名
20   カテゴリ
21   定価
22 }
23
24 受注 1--* 受注明細
25 商品マスタ 1--* 受注明細
```



```
1 lane 顧客
2 lane 営業部
3 lane 経理部
4 lane 物流部
5
6 顧客: start 注文
7 営業部: process 受注処理
8 営業部: decision 在庫確認
9 経理部: process 請求書発行
10 物流部: process 出荷手配
11 物流部: process 配送
12 顧客: end 受領
13
14 注文 -> 受注処理
15 受注処理 -> 在庫確認
16 在庫確認 -> 請求書発行 : "在庫あり"
17 請求書発行 -> 出荷手配
18 出荷手配 -> 配送
19 配送 -> 受領
```



```
1 member PM : "プロジェクトマネージャー"
2 member AP_Lead : "APリーダー"
3 member Infra_Lead : "インフラリーダー"
4 member QA_Lead : "QAリーダー"
5 member AP1 : "AP開発メンバー"
6 member AP2 : "AP開発メンバー"
7 member Infra1 : "インフラメンバー"
8 member QA1 : "QAメンバー"
9 PM -> AP_Lead
10 PM -> Infra_Lead
11 PM -> QA_Lead
12 AP_Lead -> AP1
13 AP_Lead -> AP2
14 Infra_Lead -> Infra1
15 QA_Lead -> QA1
```

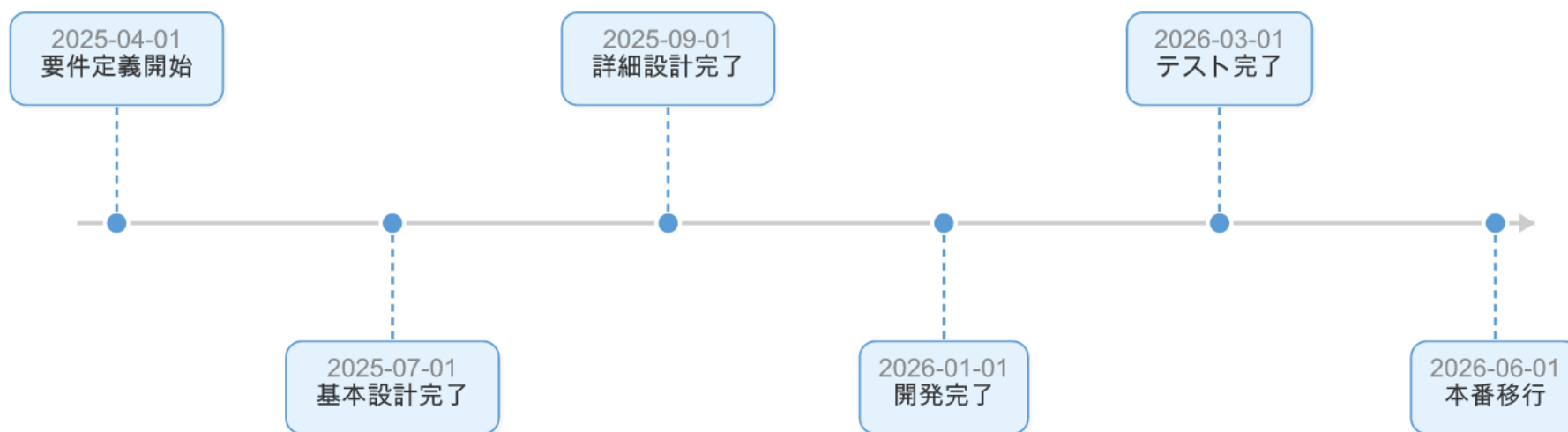


```
1  option "Spring Boot" {
2    言語: Java
3    実績: 豊富
4    学習コスト: 中
5    エコシステム: 大規模
6    保守性: 高い
7  }
8  option "Ruby on Rails" {
9    言語: Ruby
10   実績: 豊富
11   学習コスト: 低
12   エコシステム: 中規模
13   保守性: 中
14 }
15 option "Next.js" {
16   言語: TypeScript
17   実績: 増加中
18   学習コスト: 中
19   エコシステム: 大規模
20   保守性: 高い
21 }
```



どんな図が作れる？ — タイムライン

- 1 2025-04-01 : 要件定義開始
- 2 2025-07-01 : 基本設計完了
- 3 2025-09-01 : 詳細設計完了
- 4 2026-01-01 : 開発完了
- 5 2026-03-01 : テスト完了
- 6 2026-06-01 : 本番移行



- 1

x-axis "影響度：小" "影響度：大"
- 2

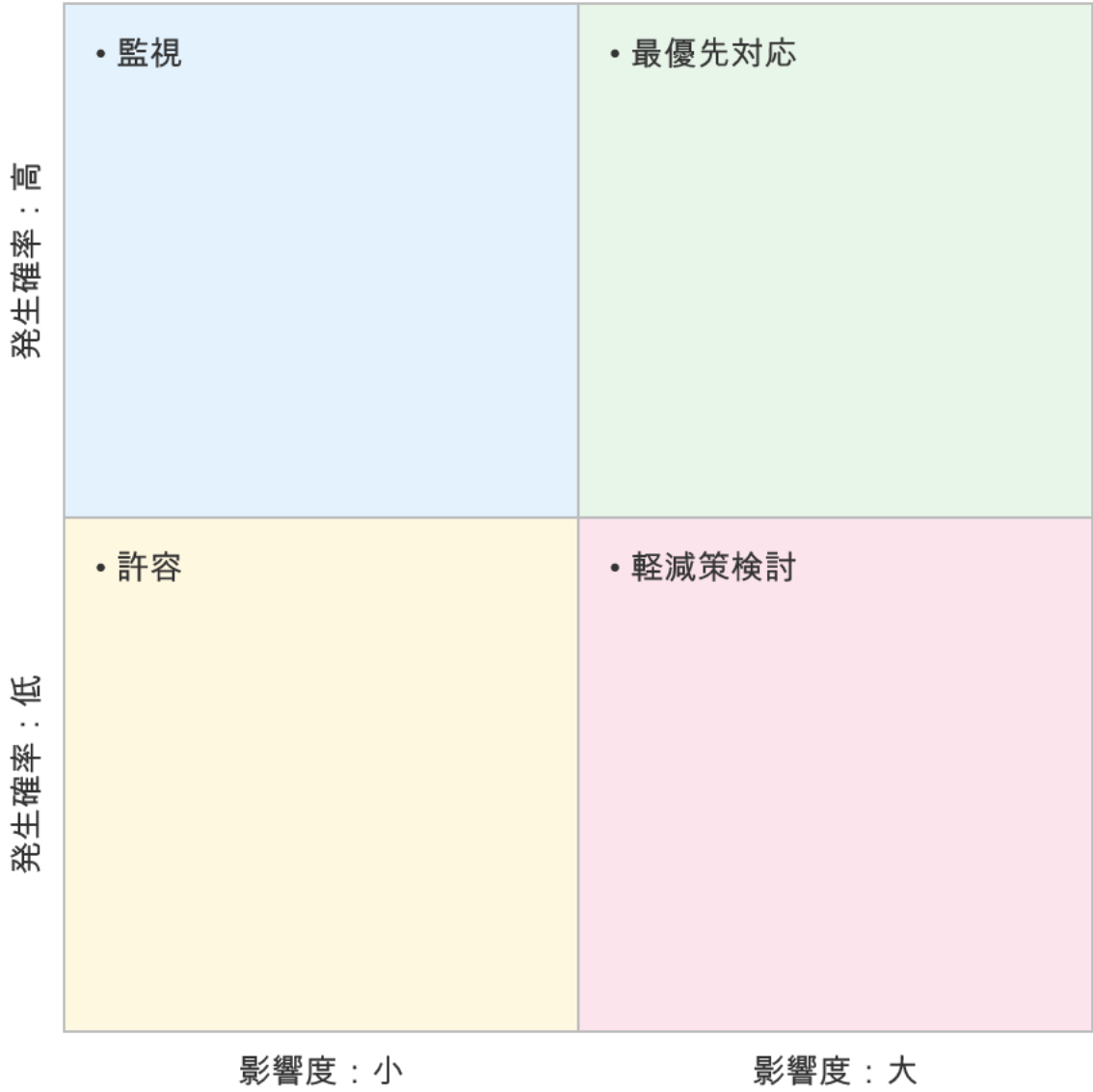
y-axis "発生確率：低" "発生確率：高"
- 3

quadrant 1 : "監視"
- 4

quadrant 2 : "最優先対応"
- 5

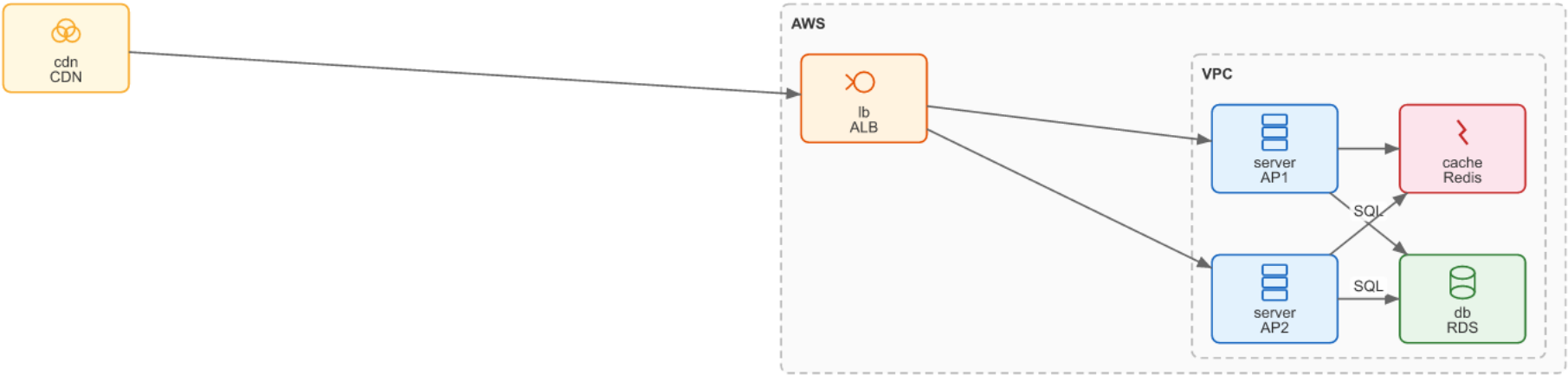
quadrant 3 : "許容"
- 6

quadrant 4 : "軽減策検討"




```
1  node CDN type=cdn
2
3  group "AWS" {
4    node ALB type=lb
5    group "VPC" {
6      node AP1 type=server
7      node AP2 type=server
8      node RDS type=db
9      node Redis type=cache
10   }
11 }
12
13 CDN -> ALB
14 ALB -> AP1
15 ALB -> AP2
16 AP1 -> RDS : "SQL"
17 AP2 -> RDS : "SQL"
18 AP1 -> Redis
19 AP2 -> Redis
```

↓ mddが変換

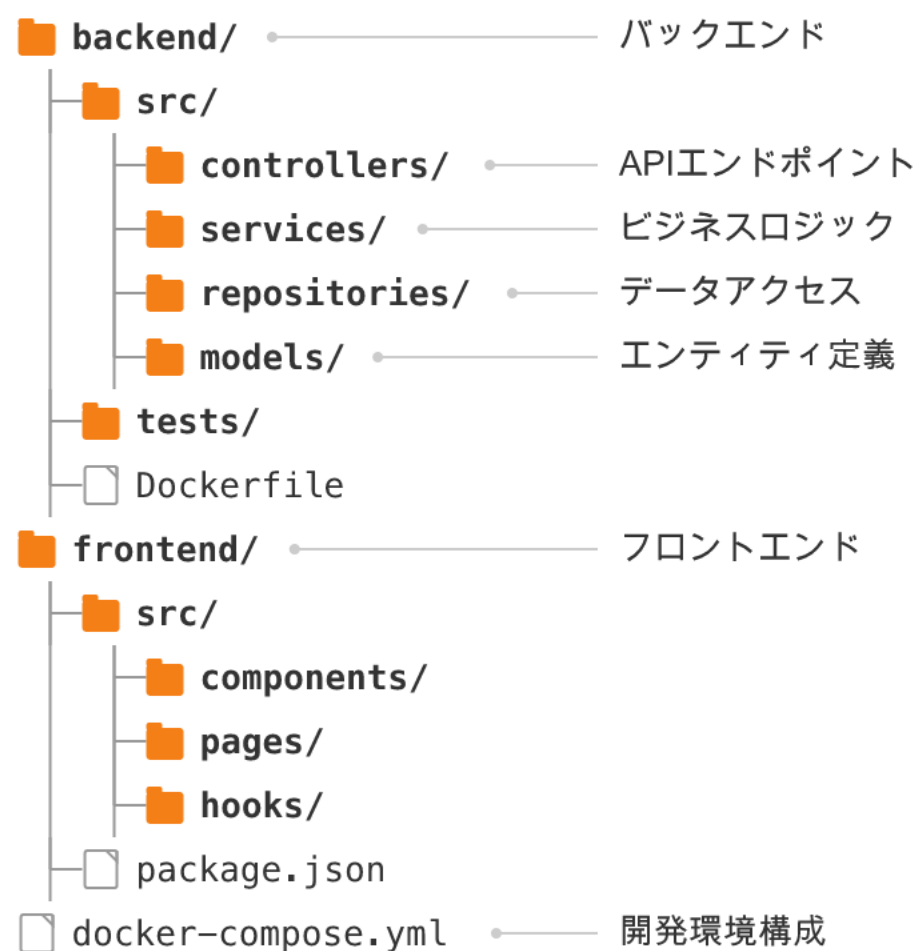



```
1 strengths {
2   チームの技術力が高い
3   既存業務知識が豊富
4 }
5 weaknesses {
6   レガシーシステムとの依存
7   テスト環境が不十分
8 }
9 opportunities {
10  クラウド移行でコスト削減
11  DXによる業務効率化
12 }
13 threats {
14  納期遅延リスク
15  要件変更の頻発
16 }
```



S - Strengths	W - Weaknesses
● チームの技術力が高い ● 既存業務知識が豊富	● レガシーシステムとの依存 ● テスト環境が不十分
O - Opportunities	T - Threats
● クラウド移行でコスト削減 ● DXによる業務効率化	● 納期遅延リスク ● 要件変更の頻発

```
1 backend/ : "バックエンド"
2 src/
3 controllers/ : "APIエンドポイント"
4 services/ : "ビジネスロジック"
5 repositories/ : "データアクセス"
6 models/ : "エンティティ定義"
7 tests/
8 Dockerfile
9 frontend/ : "フロントエンド"
10 src/
11 components/
12 pages/
13 hooks/
14 package.json
15 docker-compose.yml : "開発環境構成"
```



(おまけ) このスライドの正体

このスライド自体が、ただの Markdown テキストから自動生成されています。

```
1  title "INTRODUCTION.md"
2  ---
3  # MDD — Markdown with Diagrams
4
5  ドキュメントに図を入れたい。でも、もっと簡単に。
6
7  # 図の生成はすごく便利
8
9  ```usecase
10 actor 顧客
11 actor 管理者
12 package "認証" {
13   usecase ログイン
14 }
15 顧客 -> ログイン
```